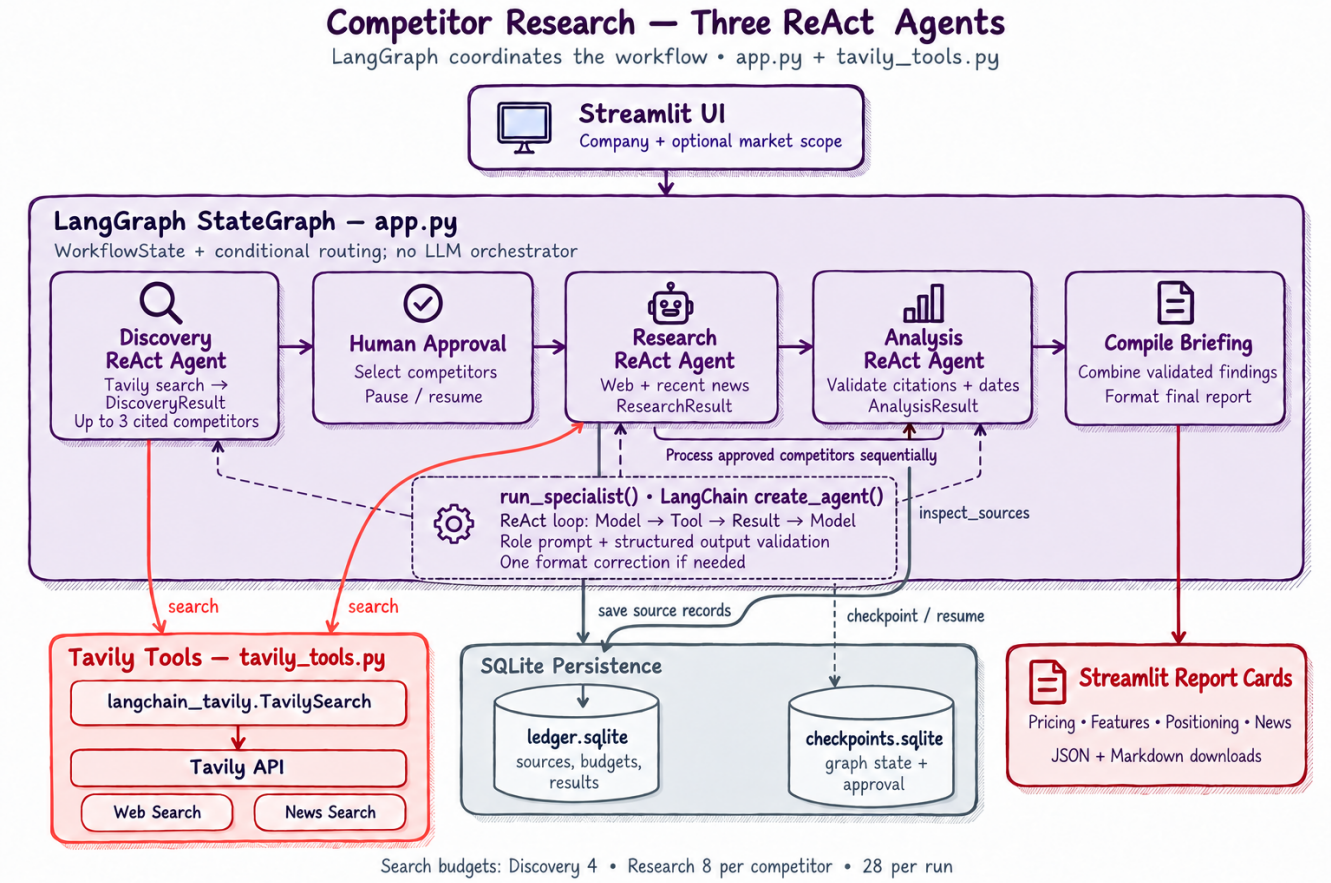
Multi-Agent Competitor Research

Project overview

This project researches a company's competitors and produces a source-linked comparison of pricing, core features, market positioning, and recent news. The user enters a company or product and an optional market scope, reviews the suggested competitors, and approves which ones to research.

The application uses **three ReAct-style specialist agents** built with LangChain, coordinated by an explicit LangGraph workflow. A Streamlit interface displays progress, comparison cards, source links, and downloadable JSON and Markdown briefings.

Architecture and workflow



Input → Discovery → Human approval → Research → Analysis → Compile briefing → Results

Architecture and workflow - responsibilities

Component	Responsibility	Output
Discovery agent	Chooses Tavily queries, infers a market scope when needed, and suggests up to three source-linked competitors.	`DiscoveryResult`: scope, rationale, competitors, source IDs, and gaps.
Research agent	Chooses web and news queries for each approved competitor and refines searches within its budget.	Saved source records and `ResearchResult`: collection summary and gaps.
Analysis agent	Calls `inspect_sources` to read that competitor's saved records; it does not search the web again.	`AnalysisResult`: pricing, features, positioning, news, source IDs, and gaps.
LangGraph	Uses `WorkflowState` and the deterministic `next_step()` router to select the next unfinished stage, pause for approval, and resume execution.	Updated state and checkpoints.
Compilation and UI	Combine validated findings, resolve citations into links, and display/export the briefing.	Streamlit cards, JSON, and Markdown.

Each specialist uses its own system prompt and tools through LangChain's `create_agent()`. Its ReAct-style loop is **model decision → tool call → tool result → next model decision**. The shared `run_specialist()` helper handles agent setup and output validation; it is not an additional agent. Competitors run sequentially.

Design decision: the original assignment requested a separate Orchestrator Agent. The current implementation intentionally replaces that LLM agent with deterministic LangGraph coordination. It therefore has three specialist agents and does not literally implement the separate fourth-agent requirement.

Technologies and project structure

- **LangChain / LangGraph:** specialist tool loops, workflow state, routing, interrupts, and checkpointing.
- **Model provider:** the current test configuration uses the OpenAI API with `gpt-4.1`. The earlier Nebius `openai/gpt-oss-120b` configuration is retained in commented code for later restoration.
- **LangChain Tavily:** primary web and news search integration.
- **Pydantic:** validates structured agent outputs.
- **Streamlit / SQLite:** interface and persistent local storage.

Application logic, prompts, schemas, graph wiring, report formatting, and UI live in `app.py`. Search tools and bounded Tavily transport live in `tavily_tools.py`. Tests are separate.

Two SQLite files have distinct roles:

- `ledger.sqlite` stores source text and URLs, competitors, approvals, search counts, research summaries, findings, reports, and activity history.
- `checkpoints.sqlite` stores LangGraph state snapshots and execution progress, including pending approval.

Some results and approvals appear in both stores. Consolidating all research data into checkpointed workflow state was considered but has not been implemented.

Datasets and sources used

No training dataset or fine-tuning is involved. Live runs build a research collection from Tavily results: source text, URLs, titles, retrieval timestamps, and available publication dates. News searches use a saved six-calendar-month window.

Offline demonstrations use synthetic Atlas Suite, Beacon Office, and Cedar Work records with simulated model responses. They demonstrate workflow behavior, not real market facts.

Discovery allows **four searches**; Research allows **eight searches per approved competitor**, with **28 logical searches per run**. Each request asks for up to five results. A transient request may retry once, so API attempts can exceed the logical search count.

Prompts used during vibe coding

Representative development instructions included:

- “Design a multi-agent market research system for competitor analysis” using Tavily, LangChain/LangGraph, and Streamlit.
- “Implement explicit top-level graph.”
- “Keep it simple. Have everything in app.py.”
- “We don't need the orchestrator agent. We can handle state management using LangGraph.”
- “Make 8 searches per competitor.”

Runtime prompts are defined separately as `DISCOVERY_PROMPT`, `RESEARCH_PROMPT`, and `ANALYSIS_PROMPT`. They specify each role’s tools, task, output contract, and rules against inventing unsupported information.

Iterations and validation

The implementation evolved from an LLM supervisor with delegation tools to explicit graph nodes, then a two-file application layout, and finally deterministic LangGraph routing. Human approval, saved-run recovery, source links, and structured outputs were retained.

Live testing exposed responses that were prose or malformed JSON instead of the required schema. A bounded correction call now supplies the exact schema and reuses collected information without repeating searches. Output still passes Pydantic validation and source/date checks.

Invalid source references are a separate issue: a schema-valid finding can still cite an unknown ID. Such findings are omitted; automatic citation repair is not currently implemented. Missing findings are not necessarily evidence that Tavily retrieval failed.

The current automated suite has **23 tests**, covering routing, approval, reopening, search budgets, schema correction, source/date validation, mocked provider calls, and Streamlit rendering. These tests use simulations and mocks; they do not establish live research accuracy.

Learnings and observations

- LangGraph can coordinate a predictable workflow without an extra supervisor model call.
- Structured JSON validation and citation validation solve different problems; both are necessary.
- More searches may improve coverage, but cannot guarantee unpublished pricing or fix incorrect citations.
- Persistent sources make it possible to inspect and recover work without repeating searches.
- Clear progress messages and concise competitor-specific notes are more useful than raw errors or long lists of optional missing details.

Reports remain source-linked AI summaries, not independently fact-checked research. A report is marked `partial` whenever recorded discovery or competitor gaps remain, including optional details. Live accuracy, cost, and latency have not been systematically benchmarked.

Running the project

Configure `OPENAI_API_KEY` and `TAVILY_API_KEY` in the local `.env`, then run:

```
.venv/bin/python -m streamlit run app.py --server.port 8504
```

Open `http://localhost:8504`. Choose **Offline sample** to demonstrate the workflow without API calls, or **Live** to retrieve current web sources.